

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Базы данных

ОТЧЕТ
по лабораторной работы №3
«Разработка NoSQL базы данных и спецификаций прикладной программы»
на тему

ПРИЮТ ДЛЯ ЖИВОТНЫХ

Студент

А.В. Губаревич

Преподаватель

С.С. Силич

Минск 2026

СОДЕРЖАНИЕ

Введение.....	3
1 Разработка конвертора.....	4
2 Разработка приложения	9
2.1 Структура berkeleydb	9
2.2 Интеграция с клиентским приложением	10
Заключение	12
Приложение А	13

ВВЕДЕНИЕ

В лабораторной работе №3 исследуются принципы построения NoSQL-хранилищ на примере СУБД BerkeleyDB. Актуальность темы связана с растущей потребностью в эффективном управлении разнородными данными вне рамок реляционного подхода.

Ключевая задача — создание конвертера для переноса данных из PostgreSQL в BerkeleyDB, требующее понимания архитектурных различий между системами. Информация преобразуется из табличной формы в модель «ключ-значение»: первичные ключи становятся идентификаторами записей, а все атрибуты сохраняются в JSON-формате. Это обеспечивает гибкость представления данных и удобство их дальнейшего использования.

В ходе работы освоены базовые средства работы с Berkeley DB через стандартные утилиты `db_load` и `db_dump`, которые вызываются из консольного приложения для формирования и проверки файловых `key/value` баз данных.

Практическая ценность работы заключается в получении навыков преобразования данных между различными форматами хранения и опыта работы с высокопроизводительными встраиваемыми СУБД.

1 РАЗРАБОТКА КОНВЕРТОРА

В таблице 1.1 приведено описание формата хранения данных, конвертированных из таблиц PostgreSQL в базы данных BerkeleyDB. В полях таблицы приведены названия столбцов таблиц PostgreSQL.

Таблица 1.1 – Формат хранения данных в BerkeleyDB

Таблица PostgreSQL	BerkeleyDB	
	Ключ	Значение (JSON)
Animal	id	{"breed", "date_of_receipt", "state_of_health", "type", "id_avinary", "id_employee"}
Aviary	id	{"square", "status", "type", "location"}
Employee	id	{"snp", "telephone", "hire_date", "post"}
Volunteer	id	{"snp", "telephone", "email", "duty"}
Adoptive_parent	id	{"snp", "telephone", "approval_status", "address"}
Medical_procedure	id	{"cost", "amount", "reason", "name"}
Feed_supply	id	{"quantity", "delivery_date", "the_supplier", "type_of_feed"}
Adoptive_animal	id	{"id_adoptive", "id_animal"}
Animal_feed_supply	id	{"id_animal", "id_feed_supply"}
Animal_volunteer	id	{"id_volunteer", "id_animal"}
Animal_medical_procedure	id	{"id_animal", "id_medical", "procedure_date"}

Примечание: для всех таблиц в качестве ключа используется первичный ключ id (автоинкрементное целое). В связующих таблицах также имеется суррогатный ключ id, поэтому используется он, а не составной ключ из внешних ключей. Такой подход упрощает реализацию и обеспечивает уникальность записей.

При необходимости составной ключ из внешних ключей также может быть использован (например, id1_id2), однако в данной работе выбран суррогатный id.

Конвертер работает по следующему алгоритму:

- Подключение к базе данных PostgreSQL.
- Получение информации о таблицах: название таблиц, название

столбцов, а также получение содержимого таблиц.

- Создание баз данных BerkeleyDB и их заполнение, создание/загрузка делается утилитой db_load.exe в BTree key/value, а проверка — db_dump.exe..

- Заккрытие открытых соединений с базами данных.

Для извлечения информации из PostgreSQL используются следующие SQL-запросы:

1. Запрос для получения имен всех таблиц схемы public
SQL-запрос:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
```

Этот запрос позволяет динамически получить список всех таблиц в схеме public. В конвертере он используется для определения набора таблиц, подлежащих преобразованию.

Реализация в конвертере:

```
def get_table_names(self):
    query = """
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'public'
    """
    with self.pg_conn.cursor() as cursor:
        cursor.execute(query)
        return [row[0] for row in cursor.fetchall()]
```

2. Запрос для получения названия столбцов таблицы
SQL-запрос:

```
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'table_name'
ORDER BY ordinal_position
```

Запрос используется для получения структуры таблицы – списка всех её столбцов в правильном порядке. Это необходимо для формирования JSON-объектов и правильного сопоставления данных

Реализация в конвертере:

```
def get_table_columns(self, table_name):
    query = """
        SELECT column_name
        FROM information_schema.columns
```

```

        WHERE table_name = %s
        ORDER BY ordinal_position
    """
    with self.pg_conn.cursor() as cursor:
        cursor.execute(query, (table_name,))
        return [row[0] for row in cursor.fetchall()]

```

3. Запрос для получения первичных ключей таблицы:

```

SELECT kcu.column_name
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
    ON tc.constraint_name = kcu.constraint_name
WHERE tc.table_name = 'table_name'
    AND tc.constraint_type = 'PRIMARY KEY'
ORDER BY kcu.ordinal_position

```

Определяет, какие столбцы образуют первичный ключ. В нашем случае для всех таблиц это столбец id. Значение первичного ключа используется как ключ записи в Berkeley DB.

Реализация:

```

def get_primary_keys(self, table_name):
    query = """
        SELECT kcu.column_name
        FROM information_schema.table_constraints tc
        JOIN information_schema.key_column_usage kcu
            ON tc.constraint_name = kcu.constraint_name
        WHERE tc.table_name = %s
            AND tc.constraint_type = 'PRIMARY KEY'
        ORDER BY kcu.ordinal_position
    """
    with self.pg_conn.cursor() as cursor:
        cursor.execute(query, (table_name,))
        result = cursor.fetchall()
        return [row[0] for row in result] if result else
['id']

```

4. Запрос для получения данных таблицы:

```

SELECT * FROM table_name

```

Универсальный запрос для извлечения всех данных из таблицы. Полученные строки затем преобразуются в JSON и сохраняются в Berkeley DB.

Реализация:

```

def get_table_data(self, table_name, columns):
    columns_str = ', '.join(columns)
    query = f'SELECT {columns_str} FROM "{table_name}"'

```

```

with self.pg_conn.cursor() as cursor:
    cursor.execute(query)
    return cursor.fetchall()

```

Аналитические запросы. Помимо служебных запросов, в приложении предусмотрены несколько аналитических запросов, которые позволяют получать полезную информацию из базы данных. Ниже приведены примеры аналитических SQL-запросов к исходной реляционной базе данных PostgreSQL, которые могут использоваться в прикладной программе для формирования отчётов и статистики. При работе с Berkeley DB аналогичная аналитика выполняется на уровне приложения путём чтения записей из соответствующих key/value баз данных и последующей агрегации данных.

5. Количество животных по типам

```

SELECT type, COUNT(*) AS count
FROM Animal
GROUP BY type
ORDER BY count DESC

```

Этот запрос показывает распределение животных по видам (кошки, собаки и т.д.). Может использоваться в статистическом разделе приложения.

6. Животные по состоянию здоровья

```

SELECT state_of_health, COUNT(*) AS count
FROM Animal
GROUP BY state_of_health
ORDER BY state_of_health

```

Позволяет оценить количество здоровых, больных, находящихся на карантине животных.

7. Список животных с информацией о вольере и ответственном сотруднике

```

SELECT a.id, a.breed, a.type, a.state_of_health,
       av.location AS aviary_location, av.status AS
aviary_status,
       e.snp AS employee_name, e.post AS employee_post
FROM Animal a
JOIN Aviary av ON a.id_aviary = av.id
JOIN Employee e ON a.id_employee = e.id
ORDER BY a.id

```

Комплексный запрос, объединяющий три таблицы для получения полной информации о животном.

8. Общая стоимость медицинских процедур по каждому животному

```

SELECT a.id, a.breed, a.type, SUM(mp.cost) AS total_cost
FROM Animal a
JOIN Animal_medical_procedure amp ON a.id = amp.id_animal
JOIN Medical_procedure mp ON amp.id_medical = mp.id
GROUP BY a.id, a.breed, a.type
ORDER BY total_cost DESC

```

Позволяет оценить затраты на лечение каждого животного.

9. Загруженность волонтеров (количество закрепленных животных)

```

SELECT v.id, v.snp, v.duty, COUNT(av.id_animal) AS
animals_count
FROM Volunteer v
LEFT JOIN Animal_volunteer av ON v.id = av.id_volunteer
GROUP BY v.id, v.snp, v.duty
ORDER BY animals_count DESC

```

Показывает, сколько животных закреплено за каждым волонтером.

Эти запросы вызываются через соответствующие пункты меню «Специальные запросы» в клиентском приложении.

2 РАЗРАБОТКА ПРИЛОЖЕНИЯ

Код программы представлен в приложении А.

2.1 Структура BerkeleyDB

В рамках реализации проекта была разработана специализированная файловая база данных на основе Berkeley DB. Архитектура системы построена по принципу прямого отображения реляционной структуры на файловую систему, где каждая таблица соответствует отдельному файлу .db в общей директории berkeley_export.

На рисунке 2.1 показана структура директории berkeley_export после успешной конвертации. Каждая таблица предметной области представлена в виде самостоятельного файла, что создает четкое логическое разделение различных типов сущностей.

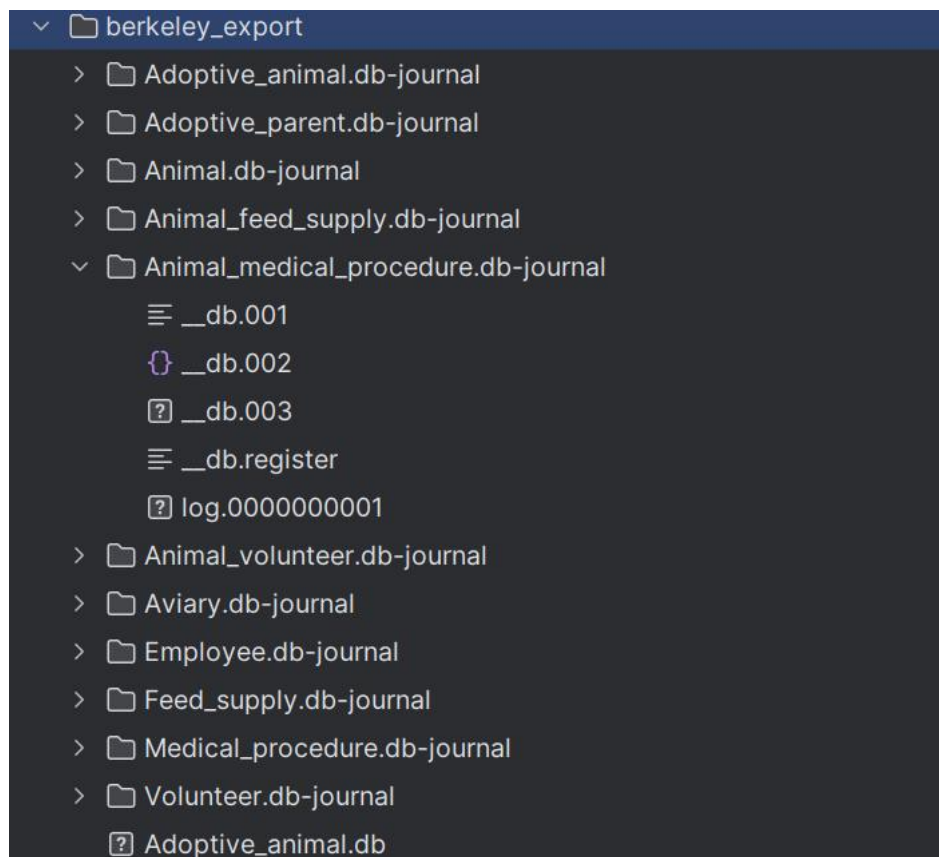


Рисунок 2.1 – Директория с базами данных Berkeley DB

```

[INFO] Конвертация таблицы: Volunteer
  Столбцы: ['id', 'snp', 'telephone', 'email', 'duty']
  Первичные ключи: ['id']
  Записей: 11
[OK] Успешно: 11 записей -> ./berkeley_export\Volunteer.db
[INFO] Dump сохранён: ./berkeley_export\dump_Volunteer.txt

[INFO] Конвертация таблицы: Aviary
  Столбцы: ['id', 'square', 'status', 'type', 'location']
  Первичные ключи: ['id']
  Записей: 10
[OK] Успешно: 10 записей -> ./berkeley_export\Aviary.db
[INFO] Dump сохранён: ./berkeley_export\dump_Aviary.txt
[INFO] Итог: таблиц найдено = 11, успешно сконвертировано = 11, файлов .db создано = 11

[OK] Конвертация завершена!
[INFO] Соединение с PostgreSQL закрыто

```

Рисунок 2.2 – Вывод консольного конвертера при создании файлов .db и итоговая сводка (количество таблиц и созданных баз).

```

1  VERSION=3
2  format=print
3  type=btree
4  db_pagesize=8192
5  HEADER=END
6  10
7  {"square": 16.5, "status": "Active", "type": "Indoor", "location": "South Wing"}
8  11
9  {"square": 35.0, "status": "Active", "type": "Isolation", "location": "\d0\9a\d0\b0\d1\80\d0\b0\d0\bd\d1\82\d0\b8\d0"}
10 2
11 {"square": 16.0, "status": "Maintenance", "type": "Indoor", "location": "South Wing"}
12 3
13 {"square": 30.0, "status": "Active", "type": "Outdoor", "location": "East Wing"}
14 4
15 {"square": 12.5, "status": "Cleaning", "type": "Indoor", "location": "West Wing"}
16 5
17 {"square": 20.0, "status": "Active", "type": "Mixed", "location": "Central Area"}
18 6
19 {"square": 18.0, "status": "Quarantine", "type": "Isolation", "location": "Medical Wing"}
20 7
21 {"square": 22.0, "status": "Active", "type": "Outdoor", "location": "Garden Area"}
22 8
23 {"square": 14.0, "status": "Renovation", "type": "Indoor", "location": "North Wing"}
24 9
25 {"square": 28.0, "status": "Active", "type": "Outdoor", "location": "Playground"}
26 DATA=END
27

```

Рисунок 2.3 – Пример результата db_dump -p для одной из баз Berkeley DB, подтверждающий структуру «ключ–значение» и JSON-формат значения.

2.2 Интеграция с клиентским приложением

Интеграция с Berkeley DB в рамках лабораторной работы реализована

через стандартные утилиты Berkeley DB, которые запускаются из приложения как внешние процессы. Создание и заполнение файловых баз данных формата key/value выполняется утилитой `db_load.exe`, которая загружает пары «ключ–значение» в B-tree базу данных. Проверка корректности результата выполняется утилитой `db_dump.exe` (режим печати), позволяющей убедиться, что в файлах `.db` присутствуют ожидаемые ключи (первичные ключи таблиц PostgreSQL) и значения в JSON-формате. Все операции конвертации выполняются автоматически при запуске консольного конвертера; на вход подаются параметры подключения к PostgreSQL и путь для экспорта, после чего приложение формирует `.db` файлы и завершает работу.

Требования к приложению: приложение должно быть консольным; при запуске автоматически выполнять конвертацию всех таблиц схемы `public` PostgreSQL в набор `.db` файлов Berkeley DB; для каждой таблицы создавать отдельную базу данных, где ключ — первичный ключ записи, значение — JSON со значениями остальных столбцов; по завершении выводить сводку (количество таблиц, количество созданных баз), а при наличии `db_dump` сохранять результат проверки в текстовые файлы дампа; при ошибках подключения или отсутствии `db_load` завершаться с сообщением об ошибке.

ЗАКЛЮЧЕНИЕ

В результате лабораторной работы был разработан и реализован комплексный проект, включающий создание NoSQL базы данных на базе BerkeleyDB и специализированного прикладного ПО. Выполненная работа позволила углубить знания о нетрадиционных подходах к хранению и обработке информации и подтвердила их практическую значимость.

Ключевым итогом стала разработка конвертера для переноса данных из реляционной СУБД PostgreSQL в формат BerkeleyDB. Предложенный алгоритм обеспечивает корректное сопоставление первичных ключей с ключами Berkeley DB и сохранение атрибутов записей в JSON-формате при переносе данных из PostgreSQL. Уникальность решения заключается в использовании первичных ключей PostgreSQL в качестве идентификаторов записей в BerkeleyDB, а также в применении JSON-сериализации для сохранения всех атрибутов исходной таблицы, что гарантирует целостность семантики данных и удобство их дальнейшего использования.

Важным практическим результатом стало освоение практики формирования и проверки Berkeley DB key/value баз данных с использованием утилит db_load/db_dump и сериализации данных в JSON. Это позволило не только эффективно реализовать конвертер, но и создать основу для построения более сложных систем управления данными. Полученный опыт подтвердил преимущества данной СУБД в сценариях, критичных к производительности и требующих минимизации издержек.

С технической точки зрения ценность работы состоит в демонстрации практических методов миграции данных между реляционными и NoSQL-системами. Эта задача особенно актуальна в условиях необходимости интеграции разнородных хранилищ. Предложенный подход может лечь в основу более сложных промышленных решений для преобразования данных.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

```
Файл convertor.py
import psycopg2
import json
import os
import subprocess
import sys
from datetime import datetime, date, time
from decimal import Decimal

DB_PARAMS = {
    'dbname': 'animal_shelter',
    'user': 'postgres',
    'password': 'atymelancholy',
    'host': 'localhost',
    'port': '5432'
}

BERKELEY_DIR = "./berkeley_export"

class JSONEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, (datetime, date)):
            return obj.isoformat()
        elif isinstance(obj, time):
            return obj.isoformat()
        elif isinstance(obj, Decimal):
            return float(obj)
        elif isinstance(obj, bytes):
            return obj.decode('utf-8', errors='ignore')
        return super().default(obj)

def _find_berkeley_tool(exe_name: str):
    possible_paths = [
        rf"C:\Program Files\Oracle\Berkeley DB 12cR1
6.0.30\bin\{exe_name}",
        rf"C:\Program Files\Oracle\Berkeley DB 12cR1
6.0.20\bin\{exe_name}",
        rf"C:\Program Files\Oracle\Berkeley DB 12cR1
6.0.18\bin\{exe_name}",
        rf"C:\Program Files (x86)\Oracle\Berkeley DB 12cR1
6.0.30\bin\{exe_name}",
        rf"C:\Program Files (x86)\Oracle\Berkeley DB 12cR1
6.0.20\bin\{exe_name}",
        rf"C:\Program Files (x86)\Oracle\Berkeley DB 12cR1
6.0.18\bin\{exe_name}",
```

```

        rf"C:\BerkeleyDB\bin\{exe_name}",
    ]
    for path in possible_paths:
        if os.path.exists(path):
            return path
    try:
        result = subprocess.run(["where.exe", exe_name],
            capture_output=True, text=True)
        if result.returncode == 0 and result.stdout.strip():
            return result.stdout.strip().splitlines()[0].strip()
    except Exception:
        pass
    return None

def get_berkeley_tools():
    db_load = _find_berkeley_tool("db_load.exe")
    db_dump = _find_berkeley_tool("db_dump.exe")
    if not db_load:
        print("[ERROR] Не найден db_load.exe. Установите Berkeley
DB (утилиты db_load/db_dump) и повторите запуск.")
        sys.exit(1)
    print(f"[OK] Найден db_load.exe: {db_load}")
    if db_dump:
        print(f"[OK] Найден db_dump.exe: {db_dump}")
    else:
        print("[WARN] db_dump.exe не найден (проверка результата
будет пропущена).")
    return db_load, db_dump

class PostgresToBerkeleyConverter:
    def __init__(self, db_load_path: str, db_dump_path: str |
None):
        self.db_load_path = db_load_path
        self.db_dump_path = db_dump_path
        self.pg_conn = None
        self.berkeley_dir = BERKELEY_DIR

    def connect_postgres(self):
        try:
            self.pg_conn = psycopg2.connect(**DB_PARAMS)
            print("[OK] Подключение к PostgreSQL успешно")
        except Exception as e:
            print(f"[ERROR] Ошибка подключения к PostgreSQL: {e}")
            sys.exit(1)

    def get_table_names(self):
        query = """
            SELECT table_name
            FROM information_schema.tables
            WHERE table_schema = 'public'
        """

```

```

        with self.pg_conn.cursor() as cursor:
            cursor.execute(query)
            return [row[0] for row in cursor.fetchall()]

def get_table_columns(self, table_name):
    query = """
        SELECT column_name
        FROM information_schema.columns
        WHERE table_name = %s
        ORDER BY ordinal_position
    """
    with self.pg_conn.cursor() as cursor:
        cursor.execute(query, (table_name,))
        return [row[0] for row in cursor.fetchall()]

def get_primary_keys(self, table_name):
    query = """
        SELECT kcu.column_name
        FROM information_schema.table_constraints tc
        JOIN information_schema.key_column_usage kcu
            ON tc.constraint_name = kcu.constraint_name
        WHERE tc.table_name = %s
            AND tc.constraint_type = 'PRIMARY KEY'
        ORDER BY kcu.ordinal_position
    """
    with self.pg_conn.cursor() as cursor:
        cursor.execute(query, (table_name,))
        result = cursor.fetchall()
        if result:
            return [row[0] for row in result]
        # fallback (если PK не определён, но столбец id есть)
        return ['id']

def get_table_data(self, table_name, columns):
    columns_str = ', '.join(columns)
    query = f'SELECT {columns_str} FROM "{table_name}"'
    with self.pg_conn.cursor() as cursor:
        cursor.execute(query)
        return cursor.fetchall()

def create_berkeley_database_dir(self):
    if not os.path.exists(self.berkeley_dir):
        os.makedirs(self.berkeley_dir)
        print(f"[INFO] {self.berkeley_dir} Создана директория:")

def convert_value(self, value):
    if value is None:
        return None
    elif isinstance(value, (datetime, date)):
        return value.isoformat()
    elif isinstance(value, time):
        return value.isoformat()

```

```

elif isinstance(value, Decimal):
    return float(value)
elif isinstance(value, (int, float, str, bool)):
    return value
elif isinstance(value, bytes):
    return value.decode('utf-8', errors='ignore')
else:
    return str(value)

@staticmethod
def _escape_for_db_load_text(s: str) -> str:
    """
    db_load -Т использует простой escape-механизм через '\'.
    Экранируем '\' и перевод строки/CR, чтобы не ломать формат
    "key\nvalue\n".
    """
    s = s.replace("\\", "\\")
    s = s.replace("\r", "\0d")
    s = s.replace("\n", "\0a")
    return s

def _make_key(self, row, columns, pk_columns):
    parts = []
    for pk_col in pk_columns:
        idx = columns.index(pk_col)
        parts.append(str(self.convert_value(row[idx])))
    return "_".join(parts)

def convert_table(self, table_name):
    print(f"\n[INFO] Конвертация таблицы: {table_name}")
    columns = self.get_table_columns(table_name)
    primary_keys = self.get_primary_keys(table_name)
    print(f"    Столбцы: {columns}")
    print(f"    Первичные ключи: {primary_keys}")

    data = self.get_table_data(table_name, columns)
    print(f"    Записей: {len(data)}")

    db_path = os.path.join(self.berkeley_dir,
f"{table_name}.db")
    temp_load_path = os.path.join(self.berkeley_dir,
f"temp_{table_name}.load.txt")

    # Готовим key/value пары для db_load (формат: key \n value
\n)
    # СТРОГО по ТЗ: ключ (PK) должен быть уникальным. Дубликаты
    PK считаем ошибкой.
    records_count = 0
    kv_by_key = {}

    try:
        for row in data:
            try:

```

```

        key_raw    =    self._make_key(row,    columns,
primary_keys)
        key = self._escape_for_db_load_text(key_raw)

        # Значение = JSON всех столбцов, КРОМЕ PK (как
в табл. 2.1 методички)
        row_dict = {}
        for i, column in enumerate(columns):
            if column in primary_keys:
                continue
            row_dict[column]
self.convert_value(row[i])

        value_json    =    json.dumps(row_dict,
ensure_ascii=False, cls=JSONEncoder)
        value
self._escape_for_db_load_text(value_json)

        if key in kv_by_key:
            raise ValueError(f"Duplicate PK detected
for key: {key_raw}")
        kv_by_key[key] = value
    except Exception as e:
        # В строгом режиме любая проблема с
ключом/данными прерывает конвертацию таблицы
        raise

    with open(temp_load_path, "w", encoding="utf-8",
newline="\n") as f:
        for key, value in kv_by_key.items():
            f.write(f"{key}\n{value}\n")
            records_count += 1

    if records_count == 0:
        print("[WARN] Нет записей для сохранения.")
        return

    # Создаём BerkeleyDB BTREE: key/value
    result = subprocess.run(
        [self.db_load_path, "-T", "-t", "btree", "-f",
temp_load_path, db_path],
        capture_output=True,
        text=True
    )

    if result.returncode == 0 and os.path.exists(db_path):
        print(f"[OK] Успешно: {records_count} записей
-> {db_path}")
    else:
        stderr = (result.stderr or "").strip()
        stdout = (result.stdout or "").strip()
        print("[ERROR] Ошибка db_load.")
        if stdout:

```

```

        print(f"    STDOUT: {stdout}")
    if stderr:
        print(f"    STDERR: {stderr}")
    return

    # Проверка результата db_dump (если доступна)
    if self.db_dump_path:
        dump_path = os.path.join(self.berkeley_dir,
f"dump_{table_name}.txt")
        dump = subprocess.run([self.db_dump_path, "-p",
db_path], capture_output=True, text=True)
        if dump.returncode == 0 and dump.stdout:
            with open(dump_path, "w", encoding="utf-8")
as df:
                df.write(dump.stdout)
            print(f"    [INFO] Dump сохранён: {dump_path}")
        else:
            print("    [WARN] Не удалось получить db_dump
для проверки.")
    finally:
        try:
            if os.path.exists(temp_load_path):
                os.remove(temp_load_path)
        except Exception:
            pass

def convert_all_tables(self):
    self.create_berkeley_database_dir()

    # Очищаем директорию от старых результатов
    if os.path.exists(self.berkeley_dir):
        for file in os.listdir(self.berkeley_dir):
            full_path = os.path.join(self.berkeley_dir, file)
            try:
                if file.endswith(".db") or
file.startswith("dump_") or file.startswith("temp_"):
                    os.remove(full_path)
                    continue
                if file.endswith(".db-journal") and
os.path.isdir(full_path):
                    # journaling каталоги от BDB SQL/SQLite
интерфейса нам не нужны
                    for root, dirs, files in
os.walk(full_path, topdown=False):
                        for name in files:
                            os.remove(os.path.join(root,
name))
                        for name in dirs:
                            os.rmdir(os.path.join(root,
name))
                    os.rmdir(full_path)
            except Exception:
                continue

```

```

        tables = self.get_table_names()
        print(f"[INFO] Найдены таблицы: {tables}")
        converted = 0
        for table in tables:
            self.convert_table(table)
            converted += 1

        try:
            db_files = [f for f in os.listdir(self.berkeley_dir)
if f.endswith(".db")]
        except Exception:
            db_files = []

        print(
            "[INFO] Итог: таблиц найдено = {found}, успешно
сконвертировано = {converted}, файлов .db создано = {dbs}".format(
                found=len(tables),
                converted=converted,
                dbs=len(db_files),
            )
        )

    def close_connections(self):
        if self.pg_conn:
            self.pg_conn.close()
            print("[INFO] Соединение с PostgreSQL закрыто")

def main():
    db_load_path, db_dump_path = get_berkeley_tools()
    converter = PostgresToBerkeleyConverter(db_load_path,
db_dump_path)
    try:
        converter.connect_postgres()
        converter.convert_all_tables()
        print("\n[OK] Конвертация завершена!")
    except Exception as e:
        print(f"\n[ERROR] Критическая ошибка: {e}")
        raise
    finally:
        converter.close_connections()

if __name__ == "__main__":
    main()

```